A FeFET-based Approximate DCIM Integrating Multiplication and OR Addition In-Bitcell

Jangseok Yu, Taehwan Kim, Seunghwa Hyun, and Jongsun Park
School of Electrical Engineering, Korea University
Seoul, Republic of Korea
{jangseok, toto9900, tmdghk5116, jongsun}@korea.ac.kr

2026 IEEE International Symposium on Circuits and Systems (ISCAS) | 979-8-3315-7769-8/26/$31.00 ©2026 IEEE | DOI: 10.1109/ISCAS66217.2026.11562639

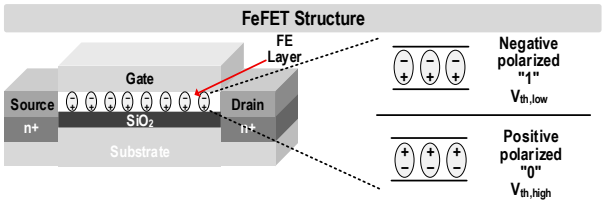
***Abstract*— Digital computing-in-memory (DCIM) preserves accuracy and is robust to process, voltage, and temperature (PVT) variations, yet the adder tree dominates macro-level area and power. To reduce the adder tree cost, DCIM with approximate adders, called approximate DCIM (ADCIM), has been proposed, but the number of addition operations in adder tree has not been reduced. In this paper, we propose a ferroelectric field-effect transistor (FeFET)-based DCIM bitcell. The proposed 2-FeFET bitcell performs two 1-bit multiplications as well as an in-bitcell OR-approximated addition simultaneously, and it can effectively replace the 1st stage adders in adder tree. As a result, when the proposed bitcell is employed for ADCIM design, nearly half of addition operations in adder tree can be performed inside 2-FeFET bitcells, thus significantly increasing area and power efficiencies. Experimental results show that the proposed ADCIM achieves 50% of area and 52.4% of power reduction over the SRAM-based DCIM baseline.**

Keywords—Computing-in-memory, FeFET, bitcell design, Approximate computing, In-bitcell OR approximated addition

I. Introduction

Computing-in-memory (CIM) has been widely explored as a way to mitigate the processor–memory data-movement bottleneck in deep neural network (DNN) processing [1]–[3]. For the implementation of CIM, various memory technologies, from SRAM to non-volatile memories such as resistive RAM (RRAM), magnetic RAM (MRAM), and ferroelectric field-effect transistor (FeFET), have been considered. However, SRAM-based CIM suffers from high leakage power for data retention and low area efficiency due to its relatively large bitcells [1]. In contrast, non-volatile memories offer very low standby power and can achieve high density [2]. Among various nonvolatile devices, FeFET, which is a gate-driven capacitive device, has been actively researched and it provides relatively low read latency due to a high I_{ON}/I_{OFF} ratio, making it particularly attractive for CIM [4]–[6].

There are two main approaches to implementing CIM using memory devices, in the analog domain and in the digital domain. Analog CIM (ACIM) performs multiply–accumulate (MAC) in analog domain and can be area and energy efficient in principle. However, peripheral circuits such as analog-to-digital converter (ADC) introduce large area and power overheads, and the analog compute path is highly sensitive to variations, which leads to significant accuracy degradation. For these reasons, recent work has increasingly favored digital CIM (DCIM), due to its robustness toward variations [1]. However, compared to ACIM, DCIM does not offer a significant reduction in hardware cost, primarily due to the large adder tree, which accounts for more than 60% of the total area and over 70% of power consumption, as well as the relatively high cost of CIM bitcells [7]. To mitigate this overhead, there have been various attempts to reduce the hardware cost. In [3], an RRAM-based DCIM bitcell has been proposed to reduce static power and bitcell area. However, the dominance of the adder tree in area and power diminishes the benefits of bitcell optimization. In addition, [7]–[10] explored replacing part of the adder tree with approximate adders to reduce adder tree cost. However, room for improvement in adder tree cost remains, as the number of additions has not been reduced in adder tree [7]–[10].



To address both the excessive adder tree overhead and the high cost bitcell, in this paper, we propose a multi-functional 2-FeFET bitcell capable of performing two partial-product computations and an in-bitcell OR-approximated addition simultaneously. Furthermore, by leveraging the proposed bitcell, we propose an ADCIM macro design in which the 1st stage adders in adder tree, accounting for nearly half of the total computation, are integrated within the 2-FeFET bitcell without addition of bitcell transistors. The integration leads to a substantial reduction in adder tree cost, thus significantly improving ADCIM macro area and energy efficiencies. The results demonstrate that the proposed design outperforms the previously proposed counterparts in terms of power, energy and area efficiencies.

The rest of this paper is organized as follows: Section II presents the background and preliminaries. Section III introduces the proposed FeFET-based ADCIM including 2-FeFET bitcell structure and its operation. Section IV presents the experimental results. Section V concludes this work.

II. Background and Preliminaries

A. FeFET Fundamentals

Fig. 1 illustrates the basic structure of a FeFET. A ferroelectric layer (FE Layer) is inserted into the gate stack of a conventional metal oxide semiconductor field-effect transistor (MOSFET) between the gate and the SiO_2 insulator. Applying appropriate gate-voltage pulses switches the ferroelectric polarization, which modulates the channel and programs the device into distinct threshold-voltage states ($V_{th,high}$ and $V_{th,low}$).

Depending on the direction of polarization, the device exhibits two operating states. Positive polarization drives the

This work was supported in part by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (RS-2024-00345481); in part by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (No. RS-2024-00405495, Plug&Play(P&P) Chiplet Integration research center). The EDA tool was supported by the IC Design Education Center(IDEC), Korea.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 30,2026 at 12:14:12 UTC from IEEE Xplore. Restrictions apply.

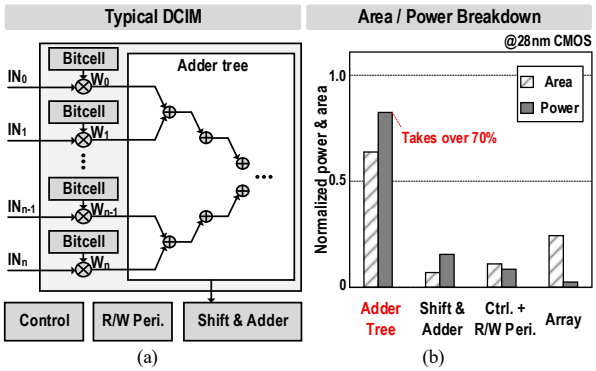


Fig. 2. (a) Typical DCIM (b) DCIM area/power breakdown [7]

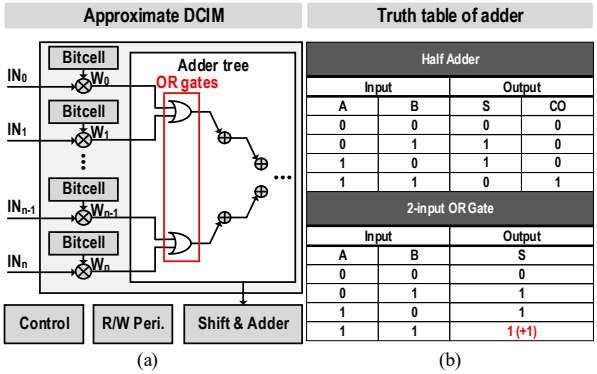


Fig. 3. (a) Approximate DCIM where the 1st stage adders in adder tree are replaced with OR gates (b) Truth tables of a half adder and a 2-input OR gate

channel into accumulation and yields $V_{th,high}$, mapped to logic '0'. Negative polarization drives inversion and yields $V_{th,low}$, mapped to logic '1'. These programmed V_{th} states are retained without supply and thus provide non-volatility. Together with low read latency and a high I_{ON}/I_{OFF} ratio, FeFET is well suited for CIM [4]–[6].

B. Approximate Digital CIM

Fig. 2(a) shows the computational flow of a typical DCIM. Weights are stored in the bitcells, partial products are generated by a multiplier, the results are accumulated by the adder tree to form partial sums, and the sums are forwarded to the shift-and-adder module. As shown in Fig. 2(b), the adder tree accounts for over 70% of total power and over 60% of area, making it the primary cost bottleneck in DCIM [7]. In the adder tree, the 1st stage adders are typically the most area-intensive. To alleviate this cost, ADCIM has been explored.

Fig. 3(a) presents an ADCIM structure where 1st stage adders are replaced with 2-input OR gates. Since the 1st stage adders are generally the most area-intensive part, replacing only this stage already yields a substantial area reduction. A half adder generally uses about 18 transistors, whereas a 2-input OR gate uses 6 transistors. As a result, replacing the 1st stage with OR gates further reduces area and power. A half adder produces a two-bit output composed of sum and carry, which requires two-bit adders in the second stage. In contrast, a 2-input OR gate produces a one-bit output, so the second stage can be simplified to one-bit adders. This also reduces the number of adders in later stages and reduces the size of adder tree.

Fig. 3(b) shows the truth tables of half adder and 2-input OR gate. For inputs (A, B) = (0,0), (0,1), and (1,0), the OR gate matches the half-adder sum exactly and differs only when A = B = 1. In that case, the half adder yields $1+1 \rightarrow 2$, whereas the OR gate approximates $1+1 \rightarrow 1$ and incurs an error of 1. Accordingly, replacing 1st stage adders with 2-input OR gates may degrade accuracy. Since OR-approximating 1st stage adders introduces error only when A = B = 1 at that stage, overall accuracy depends on how often such A = B = 1 cases occur and on the bit significance at which they occur. Occurrences on most significant bits (MSBs) induce larger errors and can propagate to subsequent stages. This approximation can degrade accuracy, and prior work reports mitigation methods. In [3], sparsity-aware fine-tuning that exploits data sparsity under an OR-approximated 1st stage is used to compensate the approximation error. In [8], accuracy loss is mitigated without retraining by proposing sparsity-aware encoding to increase MSB sparsity and in-memory swapping to suppress error patterns.

Nevertheless, structural limitations remain. In SRAM-based designs, NOR gates should be added around the bitcells to realize the multiplier [1]. In RRAM-based designs, a sense amplifier is indispensable to support multiplication through the read operation [3]. Also, prior work has not reduced the number of addition operations in the adder tree [3], [7]–[10].

In the proposed ADCIM design, 2-FeFET bitcell performs two 1-bit multiplications and an in-bitcell OR-approximated addition simultaneously. Thus, external multipliers and 1st stage adders can be eliminated. The detailed explanation on the proposed ADCIM will be presented in the following sections.

III. Proposed Idea

In this section, we demonstrate a multi-functional 2-FeFET bitcell that performs two partial-product operations and an in-bitcell OR-approximated addition concurrently. Leveraging this bitcell, an ADCIM macro is also demonstrated in which the 1st stage adders are integrated within the bitcell, achieving significant reductions in adder tree cost.

A. Proposed FeFET-based ADCIM Architecture

Fig. 4(a) shows the proposed 2-FeFET bitcell. Two FeFETs store 1-bit weights W_0 and W_1 and share a source line (SL). In CIM mode, inputs IN_0 and IN_1 are applied to the word line left (WLL) and word line right (WLR). During read and CIM operation, BL/BLb form the current return path and SL is sensed as the output node. During CIM operation, each device turns on or off according to its programmed weight and the applied input, thereby concurrently generating partial products $W_0 \cdot IN_0$ and $W_1 \cdot IN_1$. These products are OR-combined on SL. The bitcell output is as follows:

$$OUT = (W_0 \cdot IN_0) \text{ OR } (W_1 \cdot IN_1). \quad (1)$$

where W_0 and W_1 are weights programmed into the FeFETs and IN_0 and IN_1 are the input bits. Hence, the bitcell performs two 1-bit multiplications and an in-bitcell OR-approximated addition. The first stage addition is executed at the cell level, reducing the cost of the downstream adder tree.

Fig. 4(b) shows a 2×2 array layout of the proposed bitcell. Since the two FeFETs share SL, the SL routing can be overlapped inside the bitcell, reducing cell area. When multiple bitcells are placed into an array, adjacent cells allow BL-to-BL and BLb-to-BLb overlap, which reduces inter-column routing margin. Together, these two layout features improve area efficiency at both the bitcell and array level.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 30,2026 at 12:14:12 UTC from IEEE Xplore. Restrictions apply.

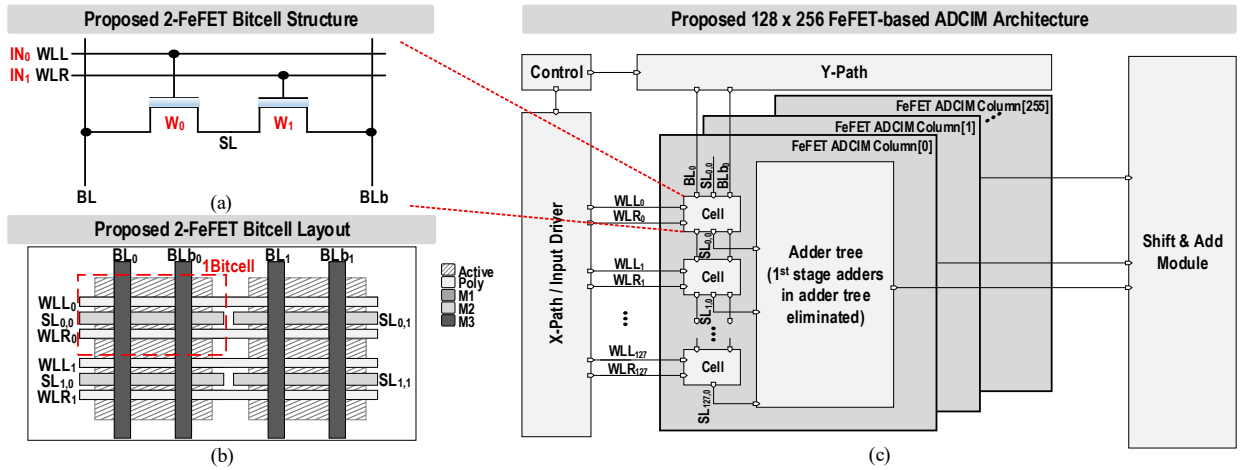
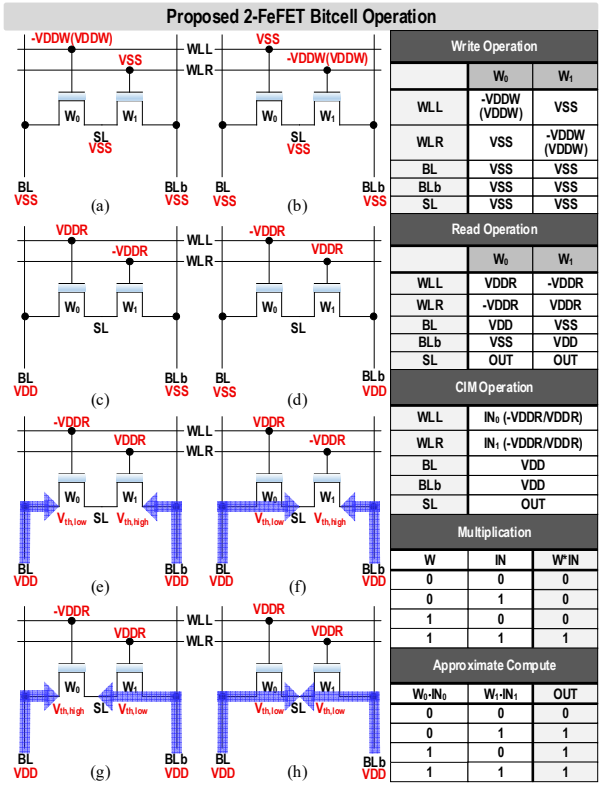


Fig. 4. (a) Proposed 2-FeFET bitcell structure (b) 2x2 bitcell layout (c) 128x256 FeFET-based ADCIM architecture (1st stage adders in adder tree eliminated)



Write Operation	W_0	W_1
WLL	-VDDW (VDDW)	VSS
WLR	VSS	-VDDW (VDDW)
BL	VSS	VSS
BLb	VSS	VSS
SL	VSS	VSS

Read Operation	W_0	W_1
WLL	VDDR	-VDDR
WLR	-VDDR	VDDR
BL	VDD	VSS
BLb	VSS	VDD
SL	OUT	OUT

CIM Operation	
WLL	IN_0 (-VDDR/VDDR)
WLR	IN_1 (-VDDR/VDDR)
BL	VDD
BLb	VDD
SL	OUT

Multiplication		
W	IN	W*IN
0	0	0
0	1	0
1	0	0
1	1	1

Approximate Compute		
W_0·IN_0	W_1·IN_1	OUT
0	0	0
0	1	1
1	0	1
1	1	1

Fig. 5. Operation of the proposed 2-FeFET bitcell (a) Write operation of the left FeFET W_0 (b) Write operation of the right FeFET W_1 (c) Read operation of the left FeFET W_0 (d) Read operation of the right FeFET W_1 (e) CIM operation that yields OUT = 0 (f) CIM operation where the left FeFET turns on and OUT = 1 (g) CIM operation where the right FeFET turns on and OUT = 1 (h) CIM operation where both FeFETs turn on and OUT = 1

Fig. 4(c) shows a 128×256 FeFET-based ADCIM architecture. Each column comprises 128 bitcells. In the proposed 2-FeFET bitcell, two FeFETs are stacked vertically as an upper and a lower device, so each bitcell stores two weight bits. Hence one column holds 256 weight bits and the total macro capacity is 64 Kb. Since the bitcell already computes the operation in (1), 1st stage adders are removed. Column-wise partial sums are then processed by the shift-and-add module to produce the final output.

Fig. 4(c) also presents the peripheral circuitry that consists of the X-path, an input driver, the Y-path, and a control module. These modules provide the required bias voltages to the memory array for write, read, and CIM operation. During CIM operation, input driver applies external inputs to WLL and WLR. During read and write operations, the Y-path asserts BL, BLb, and SL. It also sets up the sensing path. Detailed descriptions of the write, read, and CIM operations are given in Section III-B.

In summary, the proposed 2-FeFET bitcell performs two 1-bit multiplications and an OR-approximated addition in bitcell simultaneously, thereby integrating 1st stage adders into the bitcell. Since 1st stage adders are typically the most area-intensive part of the adder tree, integrating it at the cell level removes this stage entirely and adder tree begins at stage 2. From a layout perspective, SL sharing reduces the bitcell area, and BL-to-BL and BLb-to-BLb overlap across neighboring bitcells improves array-level area utilization. Consequently, the FeFET-based ADCIM architecture is realized with an adder tree whose 1st stage adders are eliminated.

B. Proposed 2-FeFET Bitcell Operation

Fig. 5 shows the write, read, and CIM operations of the proposed 2-FeFET bitcell. Fig. 5(a) and 5(b) illustrate the write operations for W_0 and W_1, respectively. Programming is performed by applying a write voltage (±VDDW) to the gate of the selected device. Applying +VDDW aligns ferroelectric polarization toward the channel, drives the device into the $V_{th,low}$ state, and programs logic 1. Applying −VDDW reverses the polarization, drives the device into the $V_{th,high}$ state, and erases to logic 0. Holding the gate of the unselected device at VSS prevents programming, enabling random access so that only one of W_0 or W_1 is updated at a time. In Fig. 5(a), the write voltage is applied to WLL while WLR is held at VSS, so only the left FeFET is programmed. In Fig. 5(b), WLL is held at VSS and the write voltage is applied to WLR, so only the right FeFET is programmed.

Fig. 5(c) and 5(d) show the read operation. The read path also supports random access to either device. In Fig. 5(c), the left FeFET is read by asserting VDDR on WLL while applying −VDDR on WLR to keep the right device disabled. Depending on the stored state of W_0, the left device turns on or remains off. When it turns on, current flows from BL to SL and SL is sensed as logic 1. When it remains off, SL is already precharged to VSS and is sensed as logic 0. Conversely, Fig. 5(d) reads the right FeFET by asserting VDDR on WLR and −VDDR on WLL so that only the right device is active.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 30,2026 at 12:14:12 UTC from IEEE Xplore. Restrictions apply.

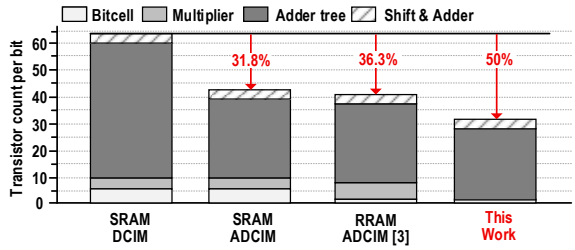


Fig. 6. In-bitcell OR addition reduces the transistor count per bit by 50%

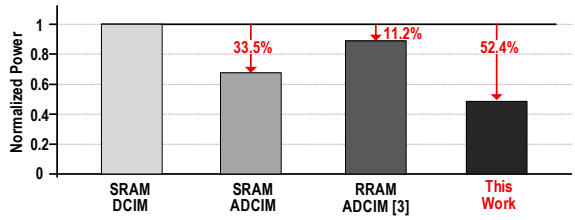


Fig. 7. In-bitcell OR addition reduces normalized power by 52.4%

Fig. 5(e)–5(h) show four representative CIM operations. Fig. 5(e) shows the case with OUT = 0. Although $W_0 = 1$, $IN_0 = 0$ biases WLL with −VDDR, left FeFET remains off. On the right, $IN_1 = 1$ but $W_1 = 0$ keeps the FeFET in the $V_{th,high}$ state, so it is off. Since no current path forms from BL or BLb to SL, SL remains precharged to VSS and is sensed as logic 0. Fig. 5(f)–5(h) show the three cases with OUT = 1. In Fig. 5(f), $W_0 = 1$ and $IN_0 = 1$ turn on the left FeFET, charging SL to VDD and yielding OUT = 1. In Fig. 5(g), $W_1 = 1$ and $IN_1 = 1$ turn on the right FeFET and OUT = 1 results. In Fig. 5(h), $W_0 = W_1 = 1$ and $IN_0 = IN_1 = 1$ turn on both FeFETs and OUT = 1.

In summary, the bitcell implements two 1-bit multiplications and the OR operation in (1). OUT is 1 if either device turns on. Each FeFET turns on only when its stored weight and the applied input are both 1, which corresponds to $W \cdot IN = 1$. Accordingly, one CIM operation completes two 1-bit multiplications and an OR-approximated addition inside the bitcell simultaneously.

IV. Experimental Results

A. Experimental Setup

We refer to [6] to model a FeFET using Verilog-A. Circuit-level simulations are performed in Cadence Virtuoso with a 28 nm CMOS technology. After functional validation of the proposed FeFET-based ADCIM, we evaluate area, power, and computational performance to assess overall efficiency. For the RRAM ADCIM baseline, we adopt LRS/HRS of 20 kΩ/200 kΩ with 10% device variation, following [3].

B. Results Analysis

For comparison, we reproduce SRAM DCIM using exact adders. The configuration consists of a 6T SRAM bitcell, a NOR-gate multiplier, and accumulation through adder tree, and the design follows [1]. We also design SRAM ADCIM by replacing the 1st stage adders with 2-input OR gates. Separately, we reproduce RRAM ADCIM based on [3], in which the 1st stage adders are also replaced with 2-input OR gates. All designs use a 28T full adder.

Fig. 6 compares transistor count per bit across four designs: SRAM DCIM, SRAM ADCIM, RRAM ADCIM, and this work. Relative to SRAM DCIM, SRAM ADCIM and RRAM ADCIM reduce the count by 31.8%–36.3%. The proposed design achieves a further 13.7%–18.2% reduction over these two approximate designs, totaling a 50% reduction versus SRAM DCIM. The primary gain comes from executing the 1st stage addition inside the bitcell and thereby eliminating the 1st stage adders. In addition, the 2-FeFET bitcell stores two weight bits and requires no external multiplier, which further improves area efficiency.

TABLE I. Comparison to Prior DCIM and ADCIM Works

Reference	ISSCC'21[1]	ISSCC'22[9]	T-CAS II'23[3]	This Work
Technology	22nm	28nm	28nm	28nm
Memory Type	SRAM	SRAM	ReRAM	FeFET
MAC Approach	Exact Adder	Approx. Adder	Approx. Adder	Approx. Adder
Bitcell Type	6T	6T	1T1R	2FeFET
Array Size	64Kb	16Kb	16Kb	64Kb
Input	1 – 8 bit	1 bit	1 bit	1 bit
Weight	4/8/12/16 bit	1 bit	1 bit	1 bit
Input toggle	18 %	25 %	18 %	25%
[1)]Normalized Throughput (GOPS/Kb)	51.6@0.72V	1552@0.9V	51.2@0.9V	512@0.7V
[2)]Normalized Efficiency (TOPS/W)	1424@0.72V	2216@0.5V	1966@0.6V	3405@0.7V
CIFAR-10 Accuracy	N/A	86.96	90.10	[3)]90.10[3]

[1)]Normalized to 1b precision and 1Kb storage.
[2)]Normalized Efficiency = Reported Efficiency × Input Bits × Weight Bits
[3)]Accuracy evaluated with the sparsity-aware fine-tuning proposed in [3]

Fig. 7 presents the normalized power comparison. In SRAM DCIM, adder tree is very large, so it dominates power. SRAM ADCIM reduces power by 33.5% relative to SRAM DCIM by reducing adder tree. The proposed FeFET-based ADCIM goes further. Since approximately half of addition operations in adder tree can be performed inside 2-FeFET bitcell by replacing 1st stage adders. Power is reduced by 52.4% compared with SRAM DCIM. Although RRAM provides the advantage of non-volatility as a resistive memory, its read operation requires relatively high energy, which lowers its overall power efficiency compared with the SRAM ADCIM. In contrast, FeFET retains non-volatility and exhibits low read energy due to high I_{ON}/I_{OFF} ratio. Therefore, the proposed ADCIM achieves superior power efficiency relative to the RRAM ADCIM.

Table I summarizes quantitative comparisons against prior RRAM- and SRAM-based DCIM/ADCIM. Compared with an SRAM-based ADCIM design [9] that adopts approximate adders, our approach offers better energy efficiency even at higher supply voltages. The proposed FeFET-based ADCIM also achieves 1.73× higher normalized energy efficiency and 10× higher normalized throughput than a RRAM-based ADCIM design [3]. This improvement is attributed to the in-bitcell 1st stage adders.

V. Conclusion

This paper presents a FeFET-based ADCIM based on a 2-FeFET bitcell. By integrating two 1-bit multiplications and an in-bitcell OR-approximated addition simultaneously, the design reduces adder tree area and power. When the proposed bitcell is employed for ADCIM design, nearly half of addition operations in adder tree are performed inside 2-FeFET bitcells, significantly increasing area and power efficiencies. As a result, compared with SRAM DCIM, the proposed ADCIM reduces transistor count per bit by 50% and power by 52.4%. Implemented in 28 nm technology, it achieves 3405 TOPS/W at 1-bit precision, a 1.73× improvement over prior work.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 30,2026 at 12:14:12 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Y. -D. Chih et al., "16.4 An 89TOPS/W and 16.3TOPS/mm2 All-Digital SRAM-Based Full-Precision Compute-In Memory Macro in22nm for Machine-Learning Edge Applications," 2021 *IEEE International Solid-State Circuits Conference (ISSCC)*, San Francisco,CA, USA, 2021, pp. 252-254

[2] T. Na, "Ternary Output Binary Neural Network With Zero-Skipping for MRAM-Based Digital In-Memory Computing," in *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 70, no. 7, pp. 2655-2659, July 2023, doi: 10.1109/TCSII.2023.3241446

[3] Y. He et al., "An RRAM-Based Digital Computing-in-Memory Macro With Dynamic Voltage Sense Amplifier and Sparse-Aware Approximate Adder Tree," in *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 70, no. 2, pp. 416-420, Feb. 2023

[4] Y. Long et al., "A Ferroelectric FET-Based Processing-in-Memory Architecture for DNN Acceleration," in *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits*, vol. 5, no. 2, pp. 113-122, Dec. 2019.

[5] A. Kar et al., "Ferroelectric Digital In-Memory Computing for Scalable, Reliable, and Efficient Similarity Computation," in *IEEE Transactions on Circuits and Systems I: Regular Papers*, doi: 10.1109/TCSI.2025.3612452.

[6] S. Deng *et al*., "A Comprehensive Model for Ferroelectric FET Capturing the Key Behaviors: Scalability, Variation, Stochasticity, and Accumulation," *2020 IEEE Symposium on VLSI Technology,* Honolulu, HI, USA, 2020, pp. 1-2

[7] K. Lee, H. Lee and J. Park, "A Design Framework of Heterogeneous Approximate DCIM-Based Accelerator for Energy-Efficient NN Processing," in *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 72, no. 8, pp. 3997-4006, Aug. 2025

[8] M. -G. Lin, J. -P. Wang, C. -Y. Chang and A. -Y. A. Wu, "Approximate Adder Tree Design with Sparsity-Aware Encoding and In-Memory Swapping for SRAM-based Digital Compute-In-Memory Macros," *2024 IEEE 6th International Conference on AI Circuits and Systems (AICAS)*, Abu Dhabi, United Arab Emirates, 2024, pp. 362-366

[9] D. Wang, C. -T. Lin, G. K. Chen, P. Knag, R. K. Krishnamurthy and M. Seok, "DIMC: 2219TOPS/W 2569F2/b Digital In-Memory Computing Macro in 28nm Based on Approximate Arithmetic Hardware," *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, San Francisco, CA, USA, 2022, pp. 266-268,

[10] C. He et al., "LSAC: A Low-Power Adder Tree for Digital Computing-in-Memory by Sparsity and Approximate Circuits Co-Design," in *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 2, pp. 852-856, Feb. 2024

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:14:12 UTC from IEEE Xplore. Restrictions apply.